

Criptografía y Seguridad

Trabajo Práctico de Implementación :

Secreto compartido en imágenes con

esteganografía



Integrantes:

Canzonieri Nicolás, 63501

Cortés Teyssier Manuel, 64159

Fernández Flores Valentina, 64142

Nacuzzi Gianna, 64006

Fecha de Entrega: 08/06/2026

Índice

1. Introducción.....	2
2. Recuperación del secreto a partir de las imágenes provistas por la cátedra.....	2
3. Cuestiones a analizar.....	3
3.1 Discusión de los aspectos relativos al documento.....	3
3.1.1 Organización formal del documento.....	3
3.1.2 Descripción del algoritmo de distribución y de recuperación.....	4
3.1.3 Notación utilizada.....	4
3.2 Imagen recuperada.....	4
3.3 Análisis sobre la seguridad en la ubicación del número de la sombra y la semilla de permutación.....	5
3.4 Criterio utilizado para ocultar las sombras k distinto de 8.....	5
3.5 Discusión de los aspectos relativos al algoritmo implementado.....	6
3.5.1 Facilidad de implementación.....	6
3.5.2 Posibilidad de extensión a imágenes a color.....	7
3.6 Dificultades en la lectura del documento y/o en la implementación.....	7
3.7 Extensiones o modificaciones posibles.....	7
3.8 Situaciones para aplicar el algoritmo.....	8
4. Conclusión.....	8

1. Introducción

El presente trabajo práctico aborda la implementación de un esquema de secreto compartido aplicado a imágenes, combinando conceptos de criptografía visual y esteganografía. El objetivo principal fue desarrollar un programa capaz de distribuir una imagen secreta en n imágenes portadoras, llamadas sombras, de forma tal que la imagen original pueda recuperarse únicamente a partir de al menos k de ellas dentro de un esquema (k, n) .

La solución implementada se basa en el algoritmo propuesto por Kuang-Shyr Wu y Tsung-Ming Lo en el paper An Efficient Secret Image Sharing Scheme. Este método extiende la idea clásica de secreto compartido de Shamir al dominio de las imágenes digitales, trabajando sobre archivos BMP de 8 bits por píxel y utilizando operaciones en aritmética modular. A su vez, las sombras generadas son ocultadas dentro de imágenes portadoras mediante técnicas de esteganografía, de manera que la información distribuida resulte poco perceptible.

El programa desarrollado permite realizar tanto la distribución como la recuperación del secreto desde línea de comandos, siguiendo la sintaxis establecida por la consigna. Además, se incorporaron validaciones sobre los parámetros de entrada, el formato de las imágenes, la capacidad de las portadoras y los límites del esquema utilizado. A partir de esta implementación, se pudo evaluar el funcionamiento del algoritmo sobre imágenes reales y analizar los principales problemas que aparecen al llevarlo a la práctica.

2. Recuperación del secreto a partir de las imágenes provistas por la cátedra

Para recuperar el secreto, se utilizaron las imágenes BMP entregadas por la cátedra como sombras del esquema de secreto compartido. Estas imágenes contienen la información necesaria para reconstruir la imagen secreta, siempre que se disponga de al menos k sombras válidas.

El comando a utilizar es:

```
./visualSSS -r -secret <nombre_archivo_salida>.bmp -k  
<valor_de_k> [-dir <directorio_de_sombras>]
```

Durante la ejecución, el programa valida las imágenes portadoras y lee la matriz de píxeles a partir del offset indicado en el encabezado BMP. Luego extrae, mediante el método de esteganografía implementado, los datos ocultos en las sombras.

A partir de los bytes reservados del archivo se obtiene el número correspondiente a cada sombra y la semilla utilizada para regenerar la tabla pseudoaleatoria.

Una vez obtenidas al menos k sombras válidas, el programa reconstruye los valores de la imagen secreta aplicando el algoritmo de recuperación basado en el esquema de Wu y Lo. Para esto se resuelven los sistemas correspondientes en aritmética módulo 257, utilizando la información provista por las sombras seleccionadas. Finalmente, se aplica la operación XOR entre los valores reconstruidos y la tabla pseudoaleatoria generada a partir de la semilla, obteniendo los píxeles originales, y se escribe la imagen recuperada en un archivo BMP de salida.

Si la recuperación fue exitosa, se obtiene el archivo BMP con el nombre indicado. La imagen recuperada fue la siguiente:



3. Cuestiones a analizar

3.1 Discusión de los aspectos relativos al documento

3.1.1 Organización formal del documento

El paper de Kuang-Shyr Wu y Tsung-Ming Lo está organizado con las siguientes secciones. En primer lugar, hay un resumen de lo que se discute en el paper en forma de Abstract. Luego, hay una introducción en la que discuten la importancia

del algoritmo del secreto compartido en imágenes y también los distintos algoritmos propuestos sobre este tema. Continúan haciendo una explicación sobre el algoritmo planteado por Shamir y luego por Thien-Lin. Posteriormente, introducen el método propuesto por ellos mismos en su paper. A continuación, muestran los resultados experimentales obtenidos con su método, mostrando la imagen secreta, las distintas sombras que se generan y luego la imagen reconstruida. Por último, listan las conclusiones a las que llegaron, destacando que su propuesta tiene un buen funcionamiento.

3.1.2 Descripción del algoritmo de distribución y de recuperación

En cuanto a la descripción del algoritmo de distribución, el paper primero parte de una imagen secreta O . A diferencia del método de Thien-Lin, los autores proponen trabajar módulo 257 en lugar de módulo 251. Antes de generar las sombras, aplican una operación XOR entre la imagen original y una tabla pseudoaleatoria R , obteniendo una imagen randomizada Q . Luego, toman los píxeles de Q de r valores y los usan como coeficientes de un polinomio de grado $r - 1$. Es decir, si se toman los píxeles $a_0, a_1, \dots, a_{r-1}$, se forma un polinomio $f(x) = a_0 + a_1x + \dots + a_{r-1}x^{(r-1)} \mod 257$. Después, ese polinomio se evalúa en distintos valores de x , desde 1 hasta n , y cada resultado se asigna a una de las n sombras. De esta manera, cada bloque de r píxeles de la imagen original genera un píxel para cada sombra. Este proceso se repite hasta procesar toda la imagen.

La descripción del algoritmo de recuperación es más corta y sigue la lógica inversa. Para reconstruir la imagen, se necesita contar con al menos r sombras. Para cada posición de las sombras, se toma un píxel de cada una, formando r puntos del polinomio original. Con esos puntos se aplica interpolación de Lagrange para recuperar los coeficientes $a_0, a_1, \dots, a_{r-1}$. Esos coeficientes corresponden a los r píxeles de la imagen randomizada Q para esa sección. Esto se repite para todas las posiciones de las sombras hasta reconstruir toda la imagen Q . Finalmente, se vuelve a aplicar XOR con la misma tabla pseudoaleatoria R , obteniendo la imagen reconstruida O' .

3.1.3 Notación utilizada

En cuanto a la notación utilizada resulta clara y además se mantiene durante todo el paper. La única inconsistencia que notamos fue cuando hace la demostración del caso $f(x) = 256$ habla del conjunto $\{a_0, a_1, a_2, \dots, a_{n-1}\}$ cuando debería ser $\{a_0, a_1, a_2, \dots, a_{r-1}\}$.

3.2 Imagen recuperada

La imagen recuperada no necesariamente es exactamente igual a la imagen original ocultada. Esto se debe a que el algoritmo trabaja módulo 257, mientras que

los píxeles de una imagen BMP de 8 bits solo pueden tomar valores entre 0 y 255. Entonces, durante la distribución puede pasar que al evaluar un polinomio en alguna sombra el resultado sea 256. Como ese valor no se puede guardar como un byte de imagen, el algoritmo propone modificar el polinomio, restando 1 al primer coeficiente no nulo, y volver a hacer la evaluación.

Esta modificación permite generar sombras con valores válidos, pero también hace que al recuperar la imagen se obtengan esos coeficientes modificados y no necesariamente los originales. En nuestra implementación seguimos este criterio. Por eso, si nunca aparece el valor 256, la imagen recuperada puede coincidir byte a byte con la original. En cambio, si aparece ese caso, algunos píxeles pueden quedar distintos. Visualmente la imagen puede seguir viéndose muy parecida, pero podrían observarse leves distorsiones.

3.3 Análisis sobre la seguridad en la ubicación del número de la sombra y la semilla de permutación

En nuestra implementación, el número de sombra se guarda en los bytes 8 y 9 del encabezado BMP, y la semilla de la tabla pseudoaleatoria se guarda en los bytes 6 y 7. Sin embargo, desde el punto de vista de seguridad, esta decisión no es muy fuerte. Cualquier persona que conozca el formato usado por el programa podría leer esos bytes del encabezado y obtener tanto el número de sombra como la semilla. En particular, la semilla no debería considerarse un secreto protegido si se guarda de esta forma. Es más una data necesaria para poder recuperar la imagen que un mecanismo de seguridad.

Si se considera otro lugar donde guardar estos datos, se podría pensar en un archivo externo asociado a las sombras, aunque eso hace más incómodo el uso porque habría que conservar ese archivo junto con las imágenes. Por otro lado, una alternativa que no cambia el lugar donde se guarda la semilla pero sí mejora su seguridad, sería cifrarla o derivarla de una clave ingresada por el usuario, para que no quede directamente expuesta en el encabezado.

3.4 Criterio utilizado para ocultar las sombras k distinto de 8

Para valores de k distintos de 8, la cantidad de bytes que componen cada sombra ya no coincide de forma exacta con la capacidad LSB básica (1 bit por pixel) de una portadora que posea las mismas dimensiones que la imagen secreta. La cantidad de bytes de cada sombra se calcula en base a la fórmula:

$$shadow_bytes = \lceil \frac{secret_pixels}{k} \rceil$$

Dado que cada byte de la sombra requiere exactamente de 8 bits y, por lo tanto, 8 píxeles de portadora para ser embebido con estenografía LSB tradicional, el espacio mínimo requerido en la portadora en píxeles es de:

$$required_pixels = shadow_bytes \times 8$$

Es decir:

$$required_pixels = \lceil \frac{secret_pixels}{k} \rceil \times 8$$

En nuestra implementación, cuando se utiliza un esquema con $k < 8$, las portadoras deben poseer dimensiones mayores que la imagen secreta para cumplir con esta capacidad de almacenamiento de LSB. En cambio, para $k > 8$, las sombras son proporcionalmente más chicas, permitiendo utilizar portadoras de dimensiones menores o del mismo tamaño.

En cuanto al problema de recuperar imágenes cuyas portadoras poseen un tamaño diferente al del secreto original, optamos por un mecanismo que almacena las dimensiones del secreto original (ancho y alto) durante la fase de distribución dentro de los campos de los header de los archivos BMP de salida, siendo: *'biClrUsed'* (bytes 46 a 49) y *'biClrImportant'* (bytes 50 a 53). Estos campos normalmente se encuentran sin uso o en desuso para imágenes en escala de grises de 8 bpp. Durante la fase de recuperación, el programa lee estas dimensiones de las portadoras y reconstruye el archivo final recortado con la resolución exacta original, logrando así que la esteganografía sea totalmente compatible y libre de pérdidas de resolución para cualquier valor de $k \in [2, 10]$.

3.5 Discusión de los aspectos relativos al algoritmo implementado

3.5.1 Facilidad de implementación

La implementación tuvo partes más fáciles y partes más difíciles. Lo más fácil fue entender la idea general del algoritmo una vez separadas sus etapas: tomar los píxeles de la imagen, generar los polinomios, obtener las sombras y luego hacer el proceso inverso para recuperar la imagen. También ayudó dividir el programa en módulos separados, porque permitió probar por separado la lectura de BMP, la esteganografía LSB, la aritmética modular y la recuperación con Lagrange.

La parte más difícil fue integrar todas esas etapas en un programa completo. No alcanzaba con implementar las fórmulas del paper, sino que había que hacer que coincidieran con el formato BMP y con la forma de ocultar los datos en las portadoras.

3.5.2 Posibilidad de extensión a imágenes a color

Extender el algoritmo para usar imágenes color de 24 bits por píxel sería posible, pero no sería automático. En las imágenes usadas en el trabajo cada píxel ocupa un byte, porque son de 8 bits. En cambio, en una imagen color de 24 bits cada píxel ocupa tres bytes, uno para cada componente de color. Por eso, habría que adaptar la forma en la que se leen los píxeles y cómo se toman los datos de la imagen.

La parte del secreto compartido podría mantenerse, porque cada componente de color también tiene valores entre 0 y 255. Es decir, se podría aplicar el algoritmo sobre esos bytes igual que se hizo con los píxeles en escala de grises. Sin embargo, habría más información para procesar, porque una imagen color tiene más datos que una imagen de 8 bits del mismo tamaño.

También habría que decidir cómo ocultar las sombras en las imágenes portadoras. Podría usarse el bit menos significativo de los bytes de color, pero habría que analizar si conviene usar los tres componentes de color o solo algunos para que la modificación no sea tan visible. En conclusión, la extensión es posible, pero requeriría modificar la lectura de imágenes, el cálculo del espacio disponible y la forma de ocultar los datos.

3.6 Dificultades en la lectura del documento y/o en la implementación

En la lectura del documento, una dificultad ocurrió en la introducción en la parte en la que se mencionan muchos trabajos previos sobre secreto compartido en imágenes. Como se nombran varios métodos distintos sin explicarlos demasiado, al principio cuesta entenderlo.

Para llevar a cabo el proyecto también fue necesario investigar por separado varios aspectos que el paper no desarrolla en detalle, especialmente el manejo de archivos BMP, la esteganografía LSB y la interpolación modular.

3.7 Extensiones o modificaciones posibles

Algunas de las mejoras ya mencionadas son la posibilidad de extender el alcance del programa para también funcionar con imágenes BMP a color, quizás con la posibilidad de aclararlo con algún parámetro para el comando. Otra mejora ya mencionada es la de un mejor ocultamiento del número de sombra y la semilla de permutación. Quizás podría explorarse la posibilidad de trabajar con imágenes de tamaños variables o de altos distintos al ancho. Podría también considerarse la opción de distribuir las sombras de una imagen en nuevos archivos BMP sin sentido en caso que no se ofrecieran otros existentes, obviamente aceptando el riesgo de que ya no habría esteganografía en juego y que imágenes así levantarían sospechas.

3.8 Situaciones para aplicar el algoritmo

El algoritmo implementado no es perfecto, y por lo discutido en 3.3 tampoco es tan seguro. Si pudiese solucionarse el problema de ocultamiento de número de sombra y semilla, y pudiese extenderse su alcance a archivos BMP a color, entonces podría considerarse como un método de enviar imágenes confidenciales a través de imágenes inconspicuas. Repartiendo las n sombras entre varios individuos o servidores, puede establecerse un sistema Shamir donde se necesita que estén de acuerdo suficientes portadores para reunir al menos k claves, y así revelar el secreto. Quizás otro uso podría ser dentro de un sistema de archivos, guardando entre imágenes inofensivas las sombras que reconstruyan a la imagen original.

4. Conclusión

En conclusión, pudimos implementar el algoritmo de secreto compartido en imágenes y combinarlo con esteganografía LSB para ocultar las sombras dentro de imágenes BMP. El trabajo mostró que el algoritmo no depende solo de la parte matemática, sino también de varios detalles de implementación, como leer correctamente el formato BMP, manejar la semilla, identificar cada sombra y recuperar los datos en el mismo orden en que fueron ocultados.

También vimos que la imagen recuperada puede no ser exactamente igual a la original en todos los casos, principalmente por el problema del valor 256 al trabajar módulo 257. Además, algunas decisiones tomadas, como guardar la semilla y el número de sombra en el encabezado, son prácticas para implementar el programa pero no son ideales desde el punto de vista de seguridad.

A pesar de estas limitaciones, el programa permite distribuir y recuperar imágenes correctamente bajo las condiciones definidas, y el trabajo deja abiertas varias mejoras posibles, como extenderlo a imágenes color o buscar una forma más segura de guardar la metadata necesaria para la recuperación.